Tech-News ARR Pipeline

> **AI assistance disclosure:** As recommended in the assignment's *Submission* section, an AI coding agent was used to assist with this exercise. All design decisions and assumptions are documented here and in the architecture document, and I am prepared to walk through every implementation detail and extend the code live.

A local Python pipeline that turns messy tech-news articles into:

- **company ARR observations over time**, in a DuckDB warehouse with bronze, silver and gold layers. Each table is exported as its own CSV.
- **a semantic and hybrid search index**, built with sentence-transformers `all-MiniLM-L6-v2` and DuckDB `FLOAT[384]` arrays.
- the required **`output/ai_articles_enriched.csv`** export.

Every ARR number traces back to the exact source row, raw string, FX rate and parser version that produced it. The design, the table grains and the production plan are in **docs/DATA_ARCHITECTURE.md**.

System requirements

- Python **3.10+** (developed on 3.11), macOS, Linux or Windows
- About 1 GB of disk for dependencies (PyTorch is pulled in by sentence-transformers) and about 2 GB of RAM
- Internet on the **first** run only, to download the embedding model (~90 MB) into the Hugging Face cache. Use `--embedder hashing` to run fully offline.

Installation

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install -e .            # installs the arr_pipeline package (src/ layout)
```

With uv: `uv venv --python 3.11 && uv pip install -r requirements.txt -e .`

Shortcuts (Makefile)

`make help` lists every target. The most common ones:

```
make install                    # create .venv + install deps
make rebuild                    # full refresh: rebuild DB + all CSV outputs
make test                       # fast test suite
make company-arr C=openai       # ARR over time for one company
make search-ai Q="regulation"   # hybrid search (AI, 2022-2024, ARR > $50M)
make sql QUERY="SELECT * FROM company_arr_latest"
```

Run the pipeline / regenerate all CSV outputs

```
python -m arr_pipeline run --full-refresh
```

This reads `data/raw/tech_news.csv` and `data/raw/company_metadata.json`, rebuilds `output/warehouse.duckdb` from scratch, and rewrites every CSV output. A plain `python -m arr_pipeline run` (without `--full-refresh`) is also safe to repeat. It is idempotent: an unchanged input adds 0 rows and embeds nothing new.

Option	Effect
`--articles a.csv b.csv`	Ingest new or additional article batches (incremental)
`--metadata m.json`	Use a different company metadata file
`--embedder hashing`	Offline, deterministic, non-semantic embeddings
`--embedder none`	Skip embeddings
`--output-dir DIR`, `--db PATH`	Relocate outputs

Outputs

```
output/
├── ai_articles_enriched.csv        # required export (124 rows)
├── unmatched_companies.csv         # company names not found in metadata
├── warehouse.duckdb                # optional DuckDB database with all tables
├── embeddings/                     # article_embeddings.npy + article_ids.json
└── warehouse/                      # one CSV per modeled table
    ├── ingest_batches.csv          bronze_articles.csv          bronze_company_metadata.csv
    ├── silver_articles.csv         silver_arr_parse.csv         silver_company_metadata.csv
    ├── silver_company_resolution.csv
    ├── dim_company.csv             fct_arr_observation.csv      article_enriched.csv
    ├── company_arr_latest.csv      company_arr_quarterly.csv    dq_issues.csv
    └── article_embeddings.csv      pipeline_runs.csv
```

Start with **`fct_arr_observation`**: one row per valid reported ARR observation, with links to the article (`article_id`) and to the exact bronze row version (`source_record_hash`). Failed or missing parses are in `silver_arr_parse` and explained in `dq_issues`.

Usage examples

CLI

```
# ARR observations over time for one company
python -m arr_pipeline company-arr openai

# Semantic search
python -m arr_pipeline search "company raises money from investors" --top-k 5

# Hybrid search: AI-related, 2022-2024, ARR > $50M, similar to a query ...
python -m arr_pipeline search "responsible AI and safety" --ai-only --min-year 2022 --max-year
2024 --min-arr 50000000

# ... or similar to an existing article
python -m arr_pipeline search --like ART0004 --ai-only --min-arr 50000000
```

Python

```
import duckdb
from arr_pipeline.embeddings import find_similar_articles, hybrid_search
from arr_pipeline.cleaning import parse_revenue, parse_published_date

find_similar_articles("new LLM product launch", top_k=5)
# -> [('ART0433', 0.4096), ('ART0713', 0.4008), ('ART0074', 0.3876), ...]  (article_id, cosine)

con = duckdb.connect("output/warehouse.duckdb", read_only=True)
hybrid_search(con, "regulatory scrutiny", ai_related=True, min_year=2022, max_year=2024,
min_arr_usd=50_000_000)

parse_revenue("€2.1B")          # RevenueParseResult(status='valid', arr_usd=2310000000,
currency='EUR', ...)
parse_published_date("04-05-2021")  # value=2021-05-04, is_ambiguous=True, alternative_date=2021-
04-05
```

SQL (`duckdb output/warehouse.duckdb`)

```
-- ARR over time for a company, with lineage
SELECT as_of_date, arr_usd, revenue_raw, currency_original, article_id, source_url
FROM fct_arr_observation WHERE company_id = 'snowflake' ORDER BY as_of_date;

-- Latest reported ARR per company, and quarterly view
SELECT company_name, latest_as_of_date, latest_arr_usd, latest_article_id FROM company_arr_latest
ORDER BY latest_arr_usd DESC;
SELECT * FROM company_arr_quarterly WHERE company_id = 'snowflake' ORDER BY year, quarter;

-- Source article for an observation
SELECT a.* FROM fct_arr_observation f JOIN article_enriched a USING (article_id) WHERE
f.observation_id = 'ARR-ART0004';

-- Filter by date, category, industry and ARR threshold
SELECT article_id, title, company_name, published_date, category, industry, arr_usd
FROM article_enriched
WHERE published_year BETWEEN 2022 AND 2024 AND industry = 'CLOUD' AND category = 'AI_ML' AND
arr_usd > 500e6;

-- Why didn't something make it into the fact table?
SELECT issue_type, severity, count(*) FROM dq_issues GROUP BY ALL ORDER BY 3 DESC;
```

The CSVs work without DuckDB too, e.g. `pd.read_csv("output/warehouse/fct_arr_observation.csv")`.

Tests

```
pytest                 # 125 fast tests (~3 s), offline, uses the hashing embedder
pytest -m slow         # + real sentence-transformers semantic test (downloads the model)
pytest --cov=arr_pipeline --cov-report=term-missing   # ~95% line coverage
```

The suite covers:

- every revenue, date, category and company-matching rule, including the spec's examples and the edge cases
- end-to-end runs on fixture batches: idempotent re-runs, incremental batches with changed records, full-refresh equivalence, lineage joins back to bronze, the AI export filter, schema drift (extra column kept, missing column rejected), grain checks, top-k determinism, hybrid search, and the CLI

Project layout

```
src/arr_pipeline/
├── config.py           paths, FX rates, thresholds, PARSER_VERSION
├── cleaning/           pure, reusable cleaning functions (no I/O)
│   ├── revenue.py      parse_revenue / parse_revenue_usd
│   ├── dates.py        parse_published_date
│   ├── categories.py   CATEGORY_MAPPING / normalize_category
│   └── companies.py    CompanyResolver, COMPANY_ALIASES
├── ingest.py           raw files -> bronze frames (strings, hashes, batch ids)
├── transform.py        bronze -> silver (enrichment, size category, company age)
├── warehouse.py        DuckDB DDL, idempotent upserts, gold SQL, grain checks, CSV export
├── embeddings.py       embedders, cache, top-k neighbours, find_similar_articles, hybrid_search
├── pipeline.py         orchestration + ai_articles_enriched export
└── __main__.py         CLI
tests/                  unit + integration tests
docs/DATA_ARCHITECTURE.md
```

Key assumptions (details in the architecture doc)

- **Every valid `revenue` value is an ARR observation** as of the article's `published_date`. Ranges use the midpoint, and the bounds are kept. Missing, `N/A` and `Not disclosed` values are classified and **never** treated as observations.
- **Fixed FX rates**: EUR×1.1, GBP×1.27, JPY÷150. Arithmetic is exact (Decimal) and the result is rounded half-up to an integer.
- **Ambiguous numeric dates**: `NN/NN/YYYY` is read as US month-first and `NN-NN-YYYY` as EU day-first. Both conventions come from the unambiguous rows in the file. All 93 ambiguous rows are flagged and keep the alternative reading.
- **Company names** go through exact, alias, normalized and guarded fuzzy matching. Multi-entity names are left unattributed unless a reviewed alias exists. The 5 companies missing from the metadata (Cohere, xAI, Mistral AI, Hugging Face, Perplexity AI) keep their ARR series but have no enrichment, and they are listed in `unmatched_companies.csv`.
- **AI filter** = `category` maps to `AI_ML` **or** `industry` maps to `AI_ML`, published 2022–2024 (inclusive), with valid ARR > $50,000,000.